



CIS 1902: Python

Week 1

Ash Fujiyama

Scheduled Topics

- Logistics
- Data Structures
- Iteration
- Functions
- If time: Memory & Copying



Logistics

- Worksheet 0 due Today @ 11:59 PM
- Homework 0 released, due two weeks from today
- Office hours this week:
 - Ash: 12-3 pm Thursday, Levine 269C
 - Foram: 6-7:30 pm Wednesday, online
- Syllabus is now up on the website

More About Strings

- Last week, we treated strings as a **sequence** of characters
 - Something that can be iterated over and indexed

More About Strings as Sequences

- Last week, we treated strings as a **sequence** of characters
 - Something that can be iterated over and indexed
- Strings can also be *sliced*
 - Instead of returning a single element in the sequence, you take a chunk
 - Syntax: `whole_string[start : end : stride]`
 - Default values:
 - start: 0
 - end: `len(whole_string)`
 - stride: 1
 - Slice does **not** include the `end` index
 - `stride` is how many elements we skip to reach the next element in our slice

Slicing Examples

```
x = "1234567890"
```

```
x[1] = "2"
```

```
x[5:] = "6789"
```

```
x[:5] = "12345"
```

```
x[1:4] = "234"
```

```
x[-4:-1] = "678"
```

```
x[::2] = "13579"
```

```
x[::-1] = "987654321"
```

Discuss: (In)valid Inputs?

Given `x` is a string of length 10:

- What happens if you try to index `x` past its last element? (aka access `x[10]`)?
- What happens if you try to access the slice `x[20:]`?
- What happens if you try to access the slice `x[:20]`?
- What happens if you try to access the slice `x[-1:-5]`?
- What happens if you try to access the slice `x[5:5]`?

String Specific Functions

- Strings are an **immutable** sequence of characters: they cannot be changed once made!

Given string variable `str`:

- `str.strip()`: removes leading and trailing whitespaces
- `str.lower()`, `str.upper()` changes the string to be lower or upper case
 - Helpful when handling case-insensitive tasks
- `str.find()` will locate any substrings that may be within the string
- Many methods to figure out what kind of character it's composed of
 - `isalpha()`: letters only
 - `isdigit()`: numbers only
 - `isspace()`: whitespace only

But Aren't Strings Immutable?

What does this print?

```
x = "ABCDEFGH"
```

```
print(x.lower())
```

But Aren't Strings Immutable?

What does this print?

```
x = "ABCDEFGH"
```

```
x.lower()
```

```
print(x)
```

“In Place” Functions

- Some functions will modify the value itself within the variable
- But some functions don't alter the input at all - they return something new!
 - If you need to access that new value later, you need to bind it to a variable

Revisiting this Example:

What does this print?

```
x = "ABCDEFGG"
```

```
print(x.lower())
```

←we are not printing x... we're printing
what is returned by calling the lower()
function

But Aren't Strings Immutable?

What does this print?

```
x = "ABCDEFGH"
```

```
x.lower()      ←return value is "thrown away"
```

```
print(x)       ←x is unchanged!
```

Lists: A Mutable Sequence

- Lists are a mutable sequence
 - Sequence => ordered
 - Can contain elements of multiple types: `x = [1, 2, "let's go"]`
 - Changing a list's element: `x[2] = 3 → x = [1, 2, 3]`
- Denoted by `[]`
- Ways to instantiate:
 - Empty list: `x = []`
 - List with content: `x = [1, 2, 3]`
 - There are more, but these are all you need for now

List Functions

- `list.sort()` : sorts the list *in place*
- `sorted(list)`: returns a copy of the list, sorted
- `list.append(e)`: adds the element `x` to the end of the list
- `list.insert(i, e)`: insert the element `e` at index `i`
- `list.count(e)`: counts how many times `e` appears in the list
- `list.index(e)`: returns the first index in the list where `e` appears
- `list.pop(i)`: remove the element at index `i`. If no `i` is provided, removes the last element.
 - Also returns the popped element
- `list.remove(e)`: remove the first occurrence of `e`
- `list.copy()`: returns a shallow copy of the list

Collection Keywords

- Can test for membership using the keywords `in` and `not in`
- We saw `in` before when we did for-loops: `for character in word:`
- One of the most popular use cases for sets is doing membership testing
- `del` can be used to delete objects
 - In Python, anything counts as an object, even basic types
 - Given list `x`, `del x[0]` deletes the first element of `x`

Filler Slide



Operators

- What an operator does can change based on the type of what it's operating on
- Some examples of operators: $+$, $-$, $*$, $/$, $==$, $=$

What would the behavior of $+$ look like when “adding” two lists?

- $[1, 2] + [\text{“hello”}, \text{“world”}]$

What would the multiplication operator do?

- $[1, 0] * 4$

Sets

- Sets are **unordered** collections of **unique** elements
- Initialization:
 - Empty set: `x = set()`
 - Set from list: `x = set([1, 2, 3, 4, 5])`
 - Set from string: `x = set("first element")`
 - Multiple elements in set: `x = {"must", "have", "content", "here"}`
- Initialization notes:
 - Cannot do `x = {}` to initialize an empty set
 - `set(A)` constructor takes in some other collection, `A`, and makes a set out of it

Set Functions

- `set.add(element)`: modifies set to include element
- `set.intersection(other)`: returns a set containing elements that were in both sets
- `set.union(other)`: returns a set containing elements that were in either set
- `set.difference(other)`: returns a set containing elements that are in `set` but not `other`

Note: you can also use operators instead of the function to do the same thing

- Intersection uses `&`
- Union uses `|`
- Difference uses `-`
- Can compute the “exclusive or” (elements either in one set or the other, but not both) using `^`

Tuples

- Tuples are immutable (ordered) sequences
- Denoted by ()
- Initialization:
 - Empty tuple: `x = tuple()`
 - Empty tuple: `x = ()`
 - Tuple with elements: `x = tuple(981, "farigiraf")`
 - Tuple without () notation: `x = 981, "farigiraf"`
 - Tuple with single element: `x = 981,`
- Short of mutating the tuple, can treat like a list

Tuple Unpacking

- How would you separate a tuple and bind its elements to individual variables?

```
x = (10, 100, 10000)
a = x[0]
b = x[1]
c = x[2]
a, b, c = x # much more elegant!
```

Dictionaries

- Mutable mapping from keys to values
 - Ordered by insertion, though this property isn't commonly utilized
- Initialization:
 - Empty dict: `x = {}`
 - Empty dict: `x = dict()`
 - Dictionary with elements: `x = { 1 : "January",
2: "February",
3: "March" }`
- Most often, you'll use the first method of initialization
- Types of key, value pairs don't need to be consistent across the dictionary
- Keys must be a hashable type: must be fully immutable

Dictionaries: Indexing

- Instead of indexing using an integer representing a position in a sequence, dictionaries index using the key

```
team = {"Bronzong": 2, "Incineroar": 1, "Golurk": 4, "Lapras": 3}
```

```
print(team["Lapras"]) # prints 3, but would raise error if map doesn't contain the key
```

- Can change the value referenced by the key, or add a new key, value pair

```
team = {"Bronzong": 2, "Incineroar": 1, "Golurk": 4, "Lapras": 3}
```

```
team["Bronzong"] = 5
```

```
team["Gothitelle"] = 2
```

```
print(team["Bronzong"]) # prints 5
```

```
print(team["Gothitelle"]) # prints 2
```

Dictionary Functions

- `dict.list()` : returns a list of the keys used in dict
- `dict.keys()`: returns a view of the dict keys
- `dict.values()`: return a view of the dict values
- `dict.items()` : returns a view of the dict key, value pairs (aka items)
- `dict.pop(key)` : removes the key, value pair from the map, returns the value of the pair
- `dict.get(key)` : will return the value if key exists, otherwise return None

Note: the `in` keyword checks for membership of a *key*, not membership of an item

dict.keys()

- Returns a view: an object that allows viewing access to the keys of the dictionary
- Will stay updated even if dict changes
- Can we say the same of `dict.list()`?
- Other things: can iterate over and do membership checks on views

Is it a valid dictionary?

1. {}
2. {"hello"}
3. {"hello" : "world", "welcome" : "everyone"}
4. {"hello" : "world", 5 : ["hello", "world"]}
5. {"hello" : "world", "hello" : "everyone"}
6. {[] : 0, [1] : 1, [5] : 1}
7. {(,): 0, (1, 2) : 5}
8. {[[3, 4], [1, 2]] : [5, 6]}

Iteration

For sequences, it's generally considered “more Pythonic” if you use this expression:

```
for item in lst:
```

```
    print(item)
```

Compare it with this one:

```
for i in range(len(lst)):
```

```
    print(lst[i])
```

But what if you wanted to access both the item **and** its index?

Iteration

For dictionaries, you can use tuple unpacking to get the key and value of each pair:

```
dict = {}
```

```
# pretend we did some stuff and added key, value pairs to dict...
```

```
for k, v in dict:
```

```
    print("key: " + k + "\n value: " + v)
```

Iteration

Use tuple unpacking + `enumerate()`!

`enumerate()`: takes in an iterable (a sequence or iterator object), and packages each value with its position in a tuple. Returns an enumerate object - an iterator that returns (index, item) tuples

- can treat like a sequence (usually how it's used)

```
for idx, item in enumerate(lst):  
    print(item)
```

Filler slide



What is the value of x at the end?

```
x = 3034
```

```
y = x
```

```
y = 3000
```

```
x = "blue"
```

```
y = x
```

```
y = "azure"
```

```
def f(a):
```

```
    a = 3000
```

```
x = 3034
```

```
f(x)
```

```
def f(a):
```

```
    a = "azure"
```

```
x = "blue"
```

```
f(x)
```

What is the value of x at the end?

```
x = [1, 2, 3, 4, 5]
y = x
y = [5, 4, 3, 2, 1]
```

```
x = [1, 2, 3, 4, 5]
y = x
y.append(6)
```

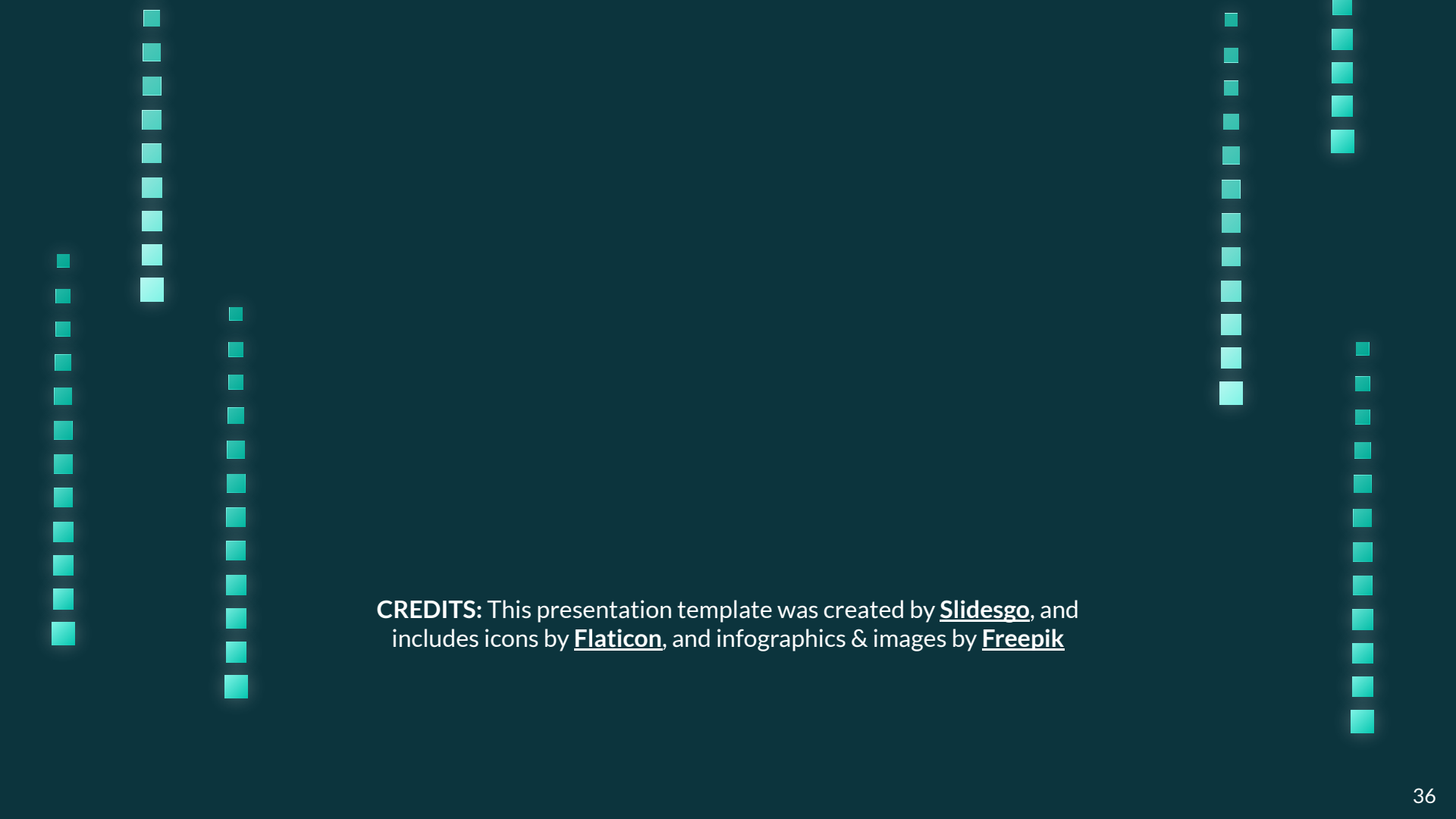
```
def f(a):
    a = [5, 4, 3, 2, 1]
x = [1, 2, 3, 4, 5]
f(x)
```

```
def f(a):
    a.append(6)
x = [1, 2, 3, 4, 5]
f(x)
```

What's actually going on

Conclusion

- Covered a lot of data structures / collections
 - Some functions and properties are shared by all collections
 - Some functions and properties are specific to a certain data structure
- Properties:
 - Mutability
 - Ordering



CREDITS: This presentation template was created by [Slidesgo](#), and includes icons by [Flaticon](#), and infographics & images by [Freepik](#)